AS OF 15 OCTOBER 2026 · WRITTEN BY JILLIENNE MORENO

Designer Workflow on the Hub

The day-to-day: sit down, build something, get it in front of the team. Every step is a sentence you type to Claude — you never open a terminal. Setting the machine up in the first place is the Contributing guide.

The day-to-day | For all five of us | Keep it open

PART 0

Before you start

This guide is the day-to-day: sit down, build something, get it in front of the team. It assumes your machine is already set up.

> **First time on this project?** Go to Contributing to the dashboard first — it covers the accounts, the clone, the dependencies and putting your name on your commits, and it has the shape of the whole loop in one picture. Come back here once *"Which branch am I on?"* gets a sensible answer. The vocabulary — `main`, branch, commit, push, rebase, merge — is in that guide too, and in Part 10 below.

When a step shows a sentence in a speech bubble, that is exactly what you type to Claude. You do not need to know a single git command; Claude knows them.

You work on your branch. Jillienne works on main.

If you ever find yourself on main, stop and tell her.

PART 1

Every time you sit down

Opening up. Open the Claude app, go to the Code tab, and make sure the folder shown is `xcel-ux-dashboard`. Start a new conversation for each piece of work. Old conversations are fine to leave.

How to talk to it

1. Type plain sentences, one step at a time. "Rebase my branch onto main" is better than three requests in one message.

2. Claude tells you what it is about to do before it does it. Read that line.

3. Before it runs a command or changes a file, a permission card appears with **Allow** and **Deny**. Click Allow when the card describes the thing you asked for. If it describes something you did not ask for, click Deny and ask what it was trying to do.

4. If Claude refuses to edit a file and names it, that refusal is correct. Those files are the Hub's own machinery and only Jillienne changes them. Tell her what you needed.

5. Typing `/` in the chat box shows the project's ready-made routines. The ones you will use are `/promote-component`, `/share` and `/promote-to-prototype`. Each one asks you questions and does the steps. Part 8 says what each one asks, what it does, and when to reach for it.

6. When Claude finishes, it summarises what it did. If anything in that summary surprises you, ask before going on.

The three questions to ask before touching anything

ASK	A GOOD ANSWER SOUNDS LIKE	IF NOT
"Which branch am I on, and do I have uncommitted changes?"	Your `feat/…` branch, and either "no changes" or a list you recognise.	On main: stop, tell Jillienne. Changes you don't recognise: ask Claude what they are first.
"Fetch origin. How far behind main is my branch?"	A number of commits. Under ten is fine for now.	Ten or more: rebase before building anything new (Part 2 shows how).
"Has anything on main changed the files I'm working on?"	"No", or a short list.	A list: read it, because that is where a conflict would come from later.

Rebasing every few days is painless. Rebasing once at the end of a month is not.

PART 2

Keeping your branch current

Main moves under you — several times a week at the moment. A rebase replays your commits on top of today's main, as if you had started this morning. It does not change your work.

Do it every few days. Rebasing a branch that is three days behind is a minute's work; rebasing one that is a month behind is an afternoon, and it is the same ten steps either way.

Do them one sentence at a time and wait for each answer.

1 **Confirm where you are.**

YOU: Which branch am I on, and do I have uncommitted changes?

You want your own `feat/…` branch. If you are somewhere else, switch to it before anything else.

2 **Save anything loose.**

If step 1 listed changes:

YOU

> Commit everything on this branch with a message that says what it is.

A rebase moves commits; it cannot move work you have not committed.

3 **Make a safety copy.**

YOU

> Create a backup branch called backup/‹my-branch›-‹today's date› pointing at where I am now, and push it.

Now nothing you do next can lose work. You can delete it next week.

4 **Do the rebase.**

YOU

> Fetch origin and rebase this branch onto origin/main. If anything conflicts, stop and show me both sides before changing anything.

Allow it. Expect it to stop if main has edited the same files you have — which is the normal case when two people are working on one surface, and exactly what step 5 is for.

Commits of yours that already reached main in another form — through someone else's promotion, say — are recognised and dropped during the rebase. Your branch coming out with fewer commits than it went in with is normal, not lost work.

5 **If it stops on a conflict.**

Claude says a file name and shows two versions of the same lines: what main has, and what your commit has. Read both. Then say one of:

YOU

> Keep my version. / Keep main's version. / Keep both — main's block first, then mine.

YOU

> Continue the rebase.

It may stop again on the next commit; do the same thing. If you cannot tell which to keep, say **"Abort the rebase"**. Your branch goes back exactly as it was. Then ask Jillienne.

6 **Confirm it worked.**

YOU

> How far behind main is my branch now, and how many commits ahead?

You want **0 behind**. The number ahead is your own work, minus anything that had already reached main.

7 **Check the build.**

YOU

> Run the build, the tests, and lint on the files I changed.

The build is the gate Netlify uses; if it fails here, your branch will not deploy. If something fails, read Claude's summary and say "Fix that and run it again." If it is in a file you never touched, stop and ask Jillienne.

8 **Push.**

YOU

> Push my rebased branch. It will need a force push with lease.

A rebase rewrites your branch's history, so a normal push is refused with a message about the branch being behind. Force-with-lease is the safe form: it refuses if someone else pushed to your branch in the meantime. This is normal on your own branch and is never done on main.

9 **Get the link.**

YOU: Give me the Netlify review link for this branch.

Netlify starts building the moment you push and takes two to four minutes. The page asks for the same password as the Hub.

10 **Look at it.**

Click through from the page a reviewer would start on. If it looks like it did before the rebase, you are done. If something that used to work is broken, that is where main and your branch disagree; tell Jillienne which page.

> After the first one, rebasing is usually just steps 4, 7 and 8. Or let Claude do it for you every weekday morning — Part 9.

PART 3

Building a change, start to finish

This is the loop you will repeat for every piece of work. Each step is one sentence to Claude.

1 **Start from main.**

YOU: Fetch origin, then make a new branch called feat/short-name from origin/main and switch to it.

Never branch from an old branch of yours; it drags the old branch's history along.

2 **Say what you want.**

Describe the change as you would to a developer sitting next to you. Name the page, the component, what it should do instead. Claude reads the code and proposes where the change goes. If it names a file you don't recognise, ask "What is that file?" before saying yes.

3 **Changing something that already exists? Use a sibling copy.**

YOU /promote-component

Name the component. It copies `StudyPaceTile.tsx` to `StudyPaceTileV2.tsx`, leaves the original untouched, and adds a flag that picks between them. This is why your branch almost never conflicts with anyone else's: nobody edits the same file.

4 **The flag is on by default on your branch.**

Claude does this when you say "behind a feature flag that is on by default on this branch". It means your branch build shows your version, and Jillienne can decide at merge whether it ships. Ask "What is the flag called?" and write it down; the review link can turn it off with `&ff=flag-name:off` if you want to show the before.

5 **Check it.**

YOU Run the type check, the tests, and lint on the files I changed.

If you added or edited anything under `public/prototypes/`, add "and the smoke suites". Red means fix before pushing: "Fix that and run it again."

6 **Commit.**

YOU Commit everything on this branch with a message that says what changed and why.

Claude writes the message in the project's style and shows it to you first.

7 **Push.**

YOU Push my branch to GitHub.

The first push on a new branch also sets it up on GitHub; Claude handles that. From then on, every push is the same sentence.

8 **Wait for Netlify.**

Two to four minutes. If you want to know rather than guess:

YOU: Check whether my branch has finished building on Netlify.

9 **Get the link.**

YOU: Give me the review link for this branch.

The shape is `https://<branch-with-dashes>--ux-design-xceldashboard.netlify.app` followed by the page: `/dashboard-rebrand?demo=1` for the dashboard, `/dashboard-rebrand?demo=1&section=compass` for Compass, `/end-to-end-demo?demo=1` for the invite flow. `?demo=1` turns on the demo controls bar.

10 **Look at it the way a reviewer would.**

Click through from the page they would start on, not straight to your component. Three bugs in September were invisible on a direct link and obvious on the first click.

11 **Keep going.**

Each further change is step 2, then `/share` again. Rebase every few days (Part 2) so the final merge is boring. When the next change is really a different piece of work, start a new branch for it rather than letting this one grow.

> **Steps 5 to 9 are one command.** `/share` runs the checks, commits, pushes, waits for the build and hands you the link, then opens the Refinement form — Part 4. The long way above is what it is doing, which is worth knowing for the day it stops halfway.

> What you never do: run the site on your own computer, edit `main`, or edit the Hub's own files. Claude will refuse and name them; ask Jillienne.

PART 4

Sharing on Refinement

Refinement is the board where work under discussion lives. Anyone adds to it, nothing on it is the source of truth, it filters by your name, and it has an archive.

A row is one chunk of work, not a branch you live on. Since 15 October rows group under the **feature** they are chunks of, newest first. When a new chunk carries on from an older one, the older one folds under "1 earlier chunk" — still there, still clickable — so a feature keeps a history you can open in a review. One long-running branch gives you a single link and no history at all.

Finish a chunk, share it, start a new branch for the next.

That is the habit the board is built around, and the reason small branches pay you back.

Adding your branch, the quick way

1. On your branch in the Code tab:

   YOU /share

   It was called `/promote-to-refinement` until 15 October, and that name still works.

2. Claude asks once — checks, commit, push, open the form? — then runs the type check and the tests, commits anything uncommitted, pushes, waits for Netlify to build your branch, works out its address and a title, and asks what feature this is a chunk of. Nothing reaches the board until you click Save.

3. In the form, **Added by** is a pick-list of the five of you. Pick your own name. The product (Atlas, Compass, Both) is already chosen from what your branch touched; change it if it is wrong.

   Feature is what this chunk is part of — start typing and the features already on the board are offered, so take one if it fits. **Replaces** appears only when that feature already has chunks, and defaults to your own newest one; its first option, *Nothing — these sit side by side*, is the right answer when two treatments should both stay up.

4 Click Save. Your row appears at the top of its feature group, showing your name, when it was last saved, and the product. Claude then offers to cut your next branch.

Adding it by hand. Open the Hub, click Refinement, click Add link, paste the review link from Part 3, type a title and a one-line note on what to look at, pick your name, pick the product, name the feature, Save.

Finding things. The pills above the list are the five names: All, Jillienne, Anjani, Eric, Afsana, Cody. Click a name to see only that person's rows; click it again to clear. There is no product pill any more; the product is the tag on the row. Rows sit under their feature heading, so looking for a piece of work means looking for its name rather than its branch.

When a row's discussion is over

YOU WANT TO	DO THIS	WHAT HAPPENS
Tidy it off the board but keep it	Click the row's ⋮, choose **Archive**	The row disappears from the board. It is stamped with when and by whom. Nothing is deleted.
See what was archived	Click **Show archive · N** at the top right	The board switches to the hidden rows. **Back to board** returns.
Bring one back	In the archive, click its ⋮, choose **Unarchive**	It is back on the board. Anyone can do this.
Delete a row that should never have existed	Click ⋮, choose **Remove**	Gone for good, after a confirmation.

Use Archive by default. Remove is for mistakes.

> **Two things to know.** The Updated tag changes only when the row is saved, not when your branch changes; after a big push, open the row and Save it again so the team sees fresh activity. And a Refinement row follows main underneath: if a component you point at changes on main, the row changes with it. Anything that must stay frozen, a user test for instance, is Jillienne's `/promote-to-testing`, not a Refinement row.

PART 5

Making your surface the prototype, and who ships it

Prototypes is the one section that means "this is the product". It holds one row per surface, every row is main's build, and each surface has an owner who decides what it should be.

SURFACE ON PROTOTYPES	OWNER	REVIEW ADDRESS
End-to-End User Experience	Jillienne	held back for now
Invite & Purchase	Afsana and Eric	`/end-to-end-demo?demo=1`
Atlas Learning	Eric	`/dashboard-rebrand?demo=1&section=dashboard`
Compass Learning	Anjani	`/dashboard-rebrand?demo=1&section=compass`
Admin Tool	Cody	placeholder until it is built

There are two separate decisions, made by two different people:

- **What the surface should be** is the owner's call, and the owner records it by running the promotion on their own branch.
- **When it reaches main** is Jillienne's call, and only she can do it.

When your branch is what your surface should become

1. Make sure the branch is current (Part 2) and the review link looks right.

2. On your branch, name your scope: "for Compass" (Anjani), "for Atlas Learning" or "for Invite & Purchase" (Eric).

   YOU /promote-to-prototype for Compass

3. Claude lists the flags your branch introduced and asks, for each, whether it should be the default. Say yes to the ones that are your design and no to anything experimental you are not ready to stand behind.

4. It commits those defaults on your branch. Nothing has reached a stakeholder yet; this is a decision written down, not a release.

5. YOU Push my branch.

6. Tell Jillienne, in words: "My Compass branch feat/… is ready to ship." Include the review link.

7. Jillienne runs ship-to-main. It refuses a branch whose owner has not run step 2, so the order matters.

8. A few minutes after she ships, your surface's row on Prototypes is your work. Its Updated tag changes to that moment.

9. Archive your Refinement row (Part 4), since the board is for things under discussion and this one is decided.

10. Start the next piece from a fresh branch off main (Part 3, step 1). Do not keep working on the shipped branch.

If you own the surface, you can also decide something should NOT go in. Say so, archive the Refinement row, and tell Jillienne; she has a routine that records why a branch was tried and declined.

PART 6

Reading the Hub

The Hub is at `ux-design-xceldashboard.netlify.app`, behind one password that Jillienne gives out. Everyone, stakeholders included, sees the same thing. Nothing inside is locked. Twelve sections in four groups:

GROUP	SECTION	WHAT IT IS FOR	WHO ADDS TO IT
Demo	**Prototypes**	The product as it stands: one row per surface, all main's build.	Owners promote; Jillienne merges.
Design & Research	**Refinement**	Chunks of work under discussion, grouped by the feature they belong to. As many as you like. Filter by name, archive when done.	Anyone, on the page.
Design & Research	**Research**	Research links and rationale.	Anyone, on the page.
Design & Research	**Resources**	Briefs, Figma files, references. Things that live elsewhere — including this guide.	Anyone, on the page.
Design & Research	**PRDs**	The product requirements the designs answer.	Anyone, on the page.
Design & Research	**Explorations**	Standalone HTML documents: outside products and ideas rebuilt on our tokens. Five today.	Jillienne adds the row; send her the file.
Dev Handoff	**Development**	Handoff notes for engineering. Empty right now.	Written with `/dev-handoff-notes`, which Jillienne runs with you.
Dev Handoff	**Done**	Finished and signed off.	Jillienne.

GROUP	SECTION	WHAT IT IS FOR	WHO ADDS TO IT
Dev Handoff	**Archive**	Components and sections removed from the project, with how to bring each back.	Jillienne.
Dev Handoff	**QA Notes**	Design-versus-build findings with captures.	Anyone, on the page.
Team	**To Do Board**	Who is working on what, as four columns a card moves across.	Anyone, on the page.
Team	**Hub Resources**	The Hub's own guides — this one, Contributing, and the stakeholder page. Each opens in a new tab with a PDF beside it.	Code, not a board: the list changes when a file is added.

What went away on 15 October, in case you look for it: the second "demo" website, the Public toggle on Refinement rows, the locked Designers group in the nav, the Design section, and the Atlas / Compass filter pills on Refinement. The old Exploration section is the Explorations row under PRDs.

PART 7

When something goes wrong

None of these lose work. Find the message, read what it means, say the sentence.

YOU SEE	IT MEANS	SAY TO CLAUDE
A refusal naming a file, like "UxDashboardPage.tsx is protected"	That file is the Hub's own machinery. Only Jillienne edits it.	Nothing. Tell Jillienne what you needed changed.
"Updates were rejected because the tip of your current branch is behind"	You rebased, so your branch's history changed, and a normal push is refused.	"Push with force-with-lease." Your branch only. Never main.
"CONFLICT (content): Merge conflict in …"	Main and your commit changed the same lines. The rebase paused for you to choose.	"Show me both sides." Then "Keep my version" / "Keep main's version" / "Keep both", then "Continue the rebase." Lost? "Abort the rebase" and ask Jillienne.
"fatal: 'main' is already checked out at …"	Someone tried to switch to main in a folder where that is not allowed. You should not be switching to main anyway.	"Switch back to my feature branch."
The build fails with lines starting `error TS`	A type error. Netlify would refuse to deploy this.	"Fix the type error and run the build again." If it is in a file you never touched, stop and ask Jillienne.
Tests fail, and the names mention `TestingGateway` on the first run only	A known slow first run when caches are cold.	"Run the tests again." If it fails twice, it is real.
The Netlify link asks for a password	Every branch build is behind the same password as the Hub.	Nothing. Enter the Hub password.

YOU SEE	IT MEANS	SAY TO CLAUDE
The Netlify link shows an old version, or "Page not found"	The build has not finished, or failed.	"Check the Netlify deploy status for my branch." If it failed, "Show me why"; usually the type error above.
"You are in 'detached HEAD' state"	Your work has no branch under it. Nothing is lost yet.	"Create a branch from here called feat/‹name› and switch to it."
Claude says it cannot run a command or lacks permission	The permission card was denied, or needs approving.	Click Allow on the card, or "Try again."
Your Refinement row shows an old Updated time	The stamp changes when the row is saved, not when the branch changes.	Open the row on the Hub, click Save.
Your commits show the author as "Claude"	Your git name was never set on this computer.	Set it, per the Contributing guide. Then: "Re-author my commits on this branch to me."
You are not sure what Claude just did	Anything.	"Summarise what you changed in the last step, file by file." Then decide.

> If none of these fit, do not try a second fix on top of the first. Copy the message, send it to Jillienne, and leave the folder as it is.

PART 8

The routines, and when to use each

A routine is a checklist Claude runs with you. Type `/` and its name in the chat box. It asks you a few questions, does the steps in order, and stops short of anything it should not decide for you.

Three of them are yours. Five belong to Jillienne; know their names so you recognise them when she mentions one.

Which one do I want?

YOU ARE THINKING	REACH FOR
"I want eyes on this one card, not the whole branch."	`/promote-component`
"I want the team to look at my branch."	`/share`
"This is what my surface should be. Ship it."	`/promote-to-prototype`, then tell Jillienne
"I'm done discussing this row."	Archive, on the row's ⋮ (no routine)
"We tried it and decided not to."	Tell Jillienne; she runs `/retire-from-refinement`
"I need a frozen link for a user test."	Tell Jillienne; she runs `/promote-to-testing`

Yours

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NEVER DOES
`/promote-component`	One component on your branch wants feedback and the rest of the branch would distract from it.	Which component (name it, or select it in the browser pane first). Which of its flag's variants to show side by side. A note telling the reviewer what to look at.	Finds the feature flag that drives the component. If there is none, shows you the flag it would add (its name, label, the one line it changes in your component) and waits for your yes. Checks the branch is pushed and built. Builds one review page with every variant stacked, each pinned. Opens Refinement's Add link form with it filled in.	Save the row (you click Add). Add a flag without showing you first. Share the whole branch.

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NEVER DOES
`/share`	A chunk of work is ready for the team to look at.	One yes, up front, covering the checks, the commit and the push. A title. A one-line note on what to look at. What feature this is a chunk of. Which product (it guesses from the files you touched: Compass files mean Compass).	Checks you are on a branch, not main. Says so if the branch has got long. Runs the type check and the tests. Commits anything uncommitted and pushes. Works out the Netlify address and waits for the build to finish. Opens Refinement's Add link form with everything filled in, your name included. Offers to cut your next branch.	Save the row (you click Add). Run on main. Commit or push without the yes it asked you for.

ROUTINE	USE IT WHEN	IT ASKS YOU	IT DOES	IT NEVER DOES
`/promote-to-prototype`	Your branch is what your surface should become, and you own that surface. Run it before you tell Jillienne it is ready.	For every flag whose default your branch changed: should this be the default, and which variant? Design-bar controls are listed separately, because those never ship as defaults.	Compares your branch with main and lists every candidate. Groups them by the Prototypes row they reach; a group that is not yours needs that owner's yes. Edits the flag defaults, runs the checks, commits on your branch. Reminds you to archive the Refinement row after the merge.	Merge anything. Add a Prototypes row. Change the default dashboard version. All three are Jillienne's.

Jillienne's: recognise, don't run

ROUTINE	WHAT IT IS FOR
`/ship-to-main`	Merging a finished branch into main. The only way anything reaches the product. It refuses a branch whose owner has not run `/promote-to-prototype`.
`/promote-to-testing`	Freezing a branch onto the separate user-test site and handing back participant and moderator links. A frozen link does not move when main does.
`/retire-from-refinement`	Closing a Refinement row properly when the work was promoted or declined: deciding what happens to the branch and writing the record of why. Different from Archive on the kebab, which only hides.
`/archive-a-feature`	Retiring a component or variant without deleting it, and writing the Archive-section row that says how to bring it back.
`/dev-handoff-notes`	Writing the handoff page engineering reads on the Development board. She runs it with you, because it asks for your words.

PART 9

The morning auto-rebase

You can have Claude rebase your branch for you every weekday morning, so Part 2 becomes something that happens while you get coffee.

Claude Code has scheduled tasks: you describe the job once, and it runs it in your project folder on a schedule, in a fresh session, and sends you a note when it is done. Two things to know first:

- **It runs while the Claude app is open.** If your laptop was closed at 8:30, it runs the next time you open the app. In practice that means "when I start my day".

- **It works on whichever branch is checked out** in your `xcel-ux-dashboard` folder. That is the branch you were on when you closed the app yesterday, which is almost always the one you want.

Set it up once

Open the Code tab on the project and paste this, exactly:

PASTE TO CLAUDE

Create a scheduled task called morning-rebase that runs every weekday at 8:30 in this project folder. Each run, do the following and nothing else:

1. Report the current branch. If it is `main`, or there is no branch checked out, stop and say so.
2. If there are uncommitted changes, stop and say so. Do not stash them and do not commit them.
3. Fetch origin. If the branch is zero commits behind origin/main, report "already current" and stop.
4. Create a local backup branch named backup/‹branch›/‹today's date› pointing at the current commit. Do not push it.
5. Rebase the branch onto origin/main. If any file conflicts, abort the rebase so the branch is exactly as it was, and report which files conflicted and in which commit.
6. If the rebase succeeded, run `npm run build` and then `npx vitest run`. If either fails, do not push. Report what failed; say that the local branch is rebased but origin still has yesterday's version.
7. If both passed, push the branch to origin with `--force-with-lease`. Never push main. Never push without the lease.
8. Finish with a six-line summary: the branch; how many commits behind it was; whether there were conflicts; whether build and tests passed; whether it pushed; and the Netlify review link for the branch.

Never run `git switch main`, never edit files, never resolve a conflict yourself, and never delete anything.

Claude confirms the schedule and the task name. Say yes.

What you see each morning

A short note in the Claude app: rebased and pushed, or stopped and why. If it says it stopped on a conflict, do Part 2 step 5 by hand. If it says checks failed, open the Code tab and say:

YOU

Show me what failed in the morning rebase.

Useful sentences

YOU WANT TO	SAY
See what is scheduled	"List my scheduled tasks."
Run it right now instead of waiting	"Run the morning-rebase task now."
Change the time	"Change morning-rebase to run at 7:45 on weekdays."
Pause it for a week	"Disable morning-rebase until next Monday."
Clean up the backups it made	"Delete my local backup/ branches older than two weeks."

> It never touches main, never writes code, and never pushes anything that did not build and pass. The worst morning it can give you is "stopped, here is why".

PART 10

Glossary

TERM	MEANING HERE
Claude Code	The Code tab in the Claude desktop app. A chat that can read and cha files in a folder, and run the project's checks for you.
Repository, repo	The project's whole history and files, on GitHub and copied to your co Ours is `jillmoreno/xcel-ux-dashboard`.
Clone	Copying the repository to your computer the first time.
main	The branch that is the product. Jillienne's.
Branch	Your own line of work, named `feat/…`. Netlify builds every branch a address.
Commit	A saved snapshot with a message. Several per day is normal.
Push	Sending commits to GitHub. Netlify builds after every push.
Fetch	Asking GitHub what has changed, without changing your files.
Rebase	Replaying your commits on top of today's main. Done on your own br often as you like.
Force push with lease	The push you need after a rebase. Safe on your branch; it refuses if so else pushed there first. Never on main.
Conflict	Main and your commit changed the same lines. You pick which wins.
Merge, ship to main	Jillienne putting a branch into main with `/ship-to-main`. The only anything reaches main.
Feature flag	A named switch in the code that picks between the old and new versi something. On by default on your branch; Jillienne decides the defau main.
Sibling copy	Copying a component to a V2 file instead of editing it, so two people n the same file. `/promote-component` does this.

TERM	MEANING HERE
Scope, owner	Each Prototypes row is a scope with an owner who decides what it sh Compass is Anjani's, Atlas Learning is Eric's, Invite & Purchase is Afsar Eric's, Admin is Cody's, End-to-End is Jillienne's.
Chunk	One piece of work, one branch, one Refinement row. Chunks of the sa feature group together on the board.
Feature (on Refinement)	What a chunk is part of, and the heading its row sits under. Free text, v board's existing features offered as you type.
Share (was promote to refinement)	Putting one chunk of your branch's work on the Refinement board for discussion. Anyone, any time.
Promote to prototype	The owner recording which flags are their surface's defaults, on their Not a release.
Branch build, review link	Your branch as a website: `https://<branch>--ux-design-xceldashboard.netlify.app/…`
`?demo=1`	Turns on the demo controls bar, so a reviewer can change learner, pro tier.
Archive (Refinement)	Hiding a row from the board, reversibly, with a stamp of who and whe
Archive (nav section)	The list of things removed from the project and how to bring each bac Different thing, same word.
Explorations	Standalone HTML documents on the Hub. Not the product.
Pull request, PR	GitHub's page for a branch that is proposed for main. Jillienne opens merges them.